

赛题理解与学习调研

欧阳易芑 2026.5.29

一、赛题原题

1. 题号：Proj 59
2. 赛题中文名称：内存受限环境的大语言模型推理优化
3. 赛题英文名称 Runtime Optimization of LLM Inference for the Memory Constraint System
4. 赛题难度等级：A
5. 赛题类型：学术型
6. 赛题维护：南开大学 宫晓利 gongxiaoli@nankai.edu.cn
7. 赛题背景：

大语言模型相关技术发展突飞猛进，但同时模型海量的参数和运行时的大量 KV cache 产生了大量的内存资源需求，使得资源受限场景难以部署。但是，大语言模型推理过程中访存行为固定，而且专家激活数量较少，为使用虚拟内存、数据预取等相关技术提供了潜在的施展空间，使得压缩运行时内存成为可能

8. 赛题任务

- 分析 LLM 推理过程的访存行为分析，包括但不限于参数加载、KV cache 使用、专家激活等。
- 使用虚拟内存的相关技术，通过按需加载、数据换出等策略减少运行时所需的物理内存量。
- 通过预取等技术掩盖 IO 延迟，提升推理性能。

9. 赛题特征

本课题注重在嵌入式设备、边缘计算设备上的大模型部署。

强调对学生的全局系统观念的考察，特别是数据访问、传输、硬件特性、计算精度等多重因素相互影响的复杂场景的处理。

10. 参考资料

FlexInfer: Breaking Memory Constraint via Flexible and Efficient Offloading for On-Device LLM Inference (EuroMLSys 2025)

11. 评审要点

评审中请注重系统的完成度，可演示性等，避免针对某一单一指标而无视其他因素的方案

二、对赛题和论文的初步理解

1. 题目拆解：

端侧边缘计算设备上缓存受限场景下运行小参数大模型，考虑大模型参数本身和 KVCache 的存储占用，要平衡 ①显存、内存、外存之间的 IO 速度 / ②模型参数量（影响模型是否聪明） / ③各级存储大小受限。

从技术上，主要关注：①LLM 底层访存的行为特征，比如参数顺序读取、KVCache 动态

增长、MoE 稀疏激活。

从工程完成上：内存压缩、IO 带宽、各级算力之间要找到最优解；同时除了代码，需要很好的可视化和很 fancy、不过时的 story。

2. FlexInfer 初步学习笔记

(1) Bg & Moti

LLM 数十亿参数远超边缘设备内存，现有方法关注模型剪枝、量化等方案，但会影响模型泛化性能和推理能力，并且缺乏灵活性，设定好超参数后所需内存大小就固定了。

而传统的 offloading，像 llama.cpp 这样的推理引擎通常用 os 的 mmap 机制，把模型存在外存里面，必要时通过 pf 加载进来。但这样就被外存 io 速度限制了，推理速度很低。在这种背景下提出了 FlexInfer。

(2) Ds

他们的 ds 中把外存作为内存的延伸，在用户指定的任何内存预算下最大化推理吞吐。主要是三个部件的技术：

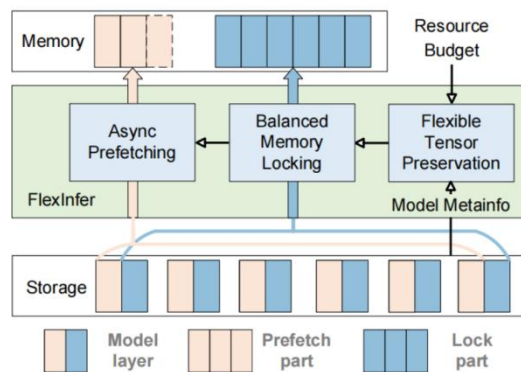


Figure 1. FlexInfer architecture.

① Async Prefetch 异步预取：

传统的同步预取中，CPU 要等 IO 结束之后才开始计算，等于耗费了计算+IO 的时间，复杂度 $O(m+n)$ 。所以他们引入：

并行化：通过多线程并行化处理 IO 和 CPU，计算线程在处理前一层的时候，IO 线程可以提前加载后面基层的参数（张量）。时间复杂度降到了 $O(\max(m,n))$

降低内存占用：LLM 生成 token 的时候参数只会被访问一次，所以内存局部性根本不存在。所以他们就参数用完之后马上释放，所以内存占用也缩小到了预取窗口大小。

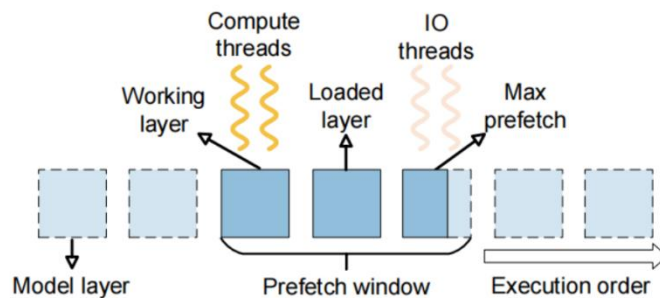


Figure 2. Asynchronous prefetching.

② Balanced Mem Lock 均衡内存锁定：

如果设备有一定的空余内存，系统就应该把部分参数锁定在内存里面来减少 IO 负担。不平衡锁定时，前几层锁在内存但后几层依赖 IO，这样就每层处理速度差距很大，计算和 IO 线程互相等待，破坏异步预取的效率。

因此，他们把每层的参数分成两部分，一部分锁定内存，另一部分靠 IO 动态预取，确保 IO 和计算负载均衡。（读到这里应该有个疑问，分类依据是什么？为什么不是三部分？比例是多少？我当时也有这个疑问 hhh，所以第三部分其实就解决这个问题。）

③Flexible Tensor Preserve 灵活张量保留：

Transformer 里面 Attention 张量和 FFN 张量比例一般 1:3，所以他们采用启发式算法来智能决定（读到这里又应该有个疑问，这个启发算法计算过程是否会增大计算负担、引入额外延迟？）那些张量锁在内存：

内存充足时：优先把比较大的 FFN 张量锁在内存里面，减少内存碎片，保持各层之间剩余需要预取的 IO 负载差距最小。

内存受限时：有限保留 attention 张量，比较小，可以把尽可能多的层的塞进来，减少 io 次数，让 io 专门处理 FFN 大块连续读取。

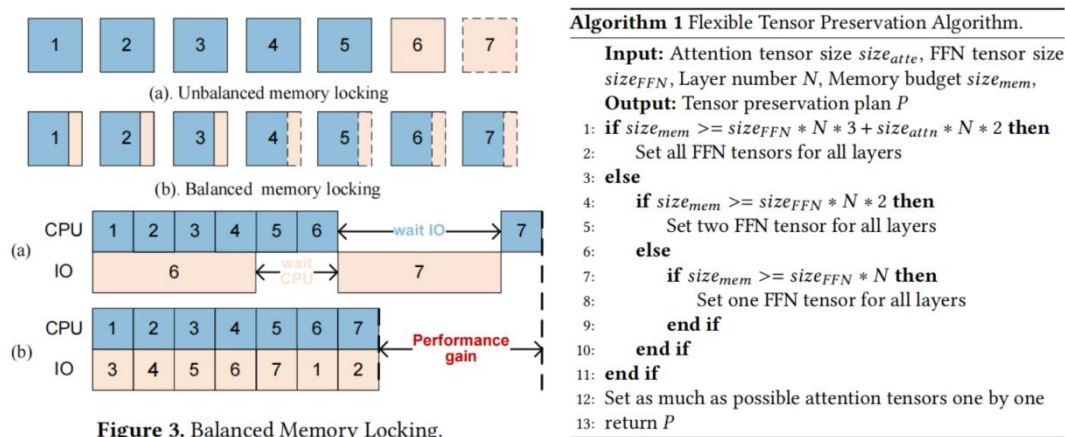


Figure 3. Balanced Memory Locking.

(3) Eval

受限环境下，比 baseline 的 mmap 实现了 10.6-12.5 倍提升。Ablation 证明三个部分分别有用。

(4) 我读完之后的一些问题

①在 CPU+内存的端侧环境和我以前在高算力 GPU+显存服务器上的区别是什么？

FlexInfer 是基于著名开源项目 llama.cpp 开发的。llama.cpp 是用 C/C++ 纯手写的推理引擎，它没有使用依赖 GPU 的 CUDA 算子，而是使用了一套名为 GGML 的张量数学库。它通过极致优化 CPU 的 SIMD（单指令多数据流）指令集（例如针对 x86 架构的 AVX2/AVX-512，或针对 ARM 架构的 NEON）来实现高效的矩阵乘法（MatMul）。虽然单次计算绝对速度不如 GPU，但在低并发、单 Batch 的端侧场景下，CPU 算力通常是够用的。

不考虑 GPU 的时候，主要就是要掩盖 CPU 等硬盘 IO 的延迟。

②启发式算法？

经验性算法，用低代价在模型加载初始化阶段算一次就行了，O(1)级别。

(5) 复现相关技术栈

C++/llama.cpp/GGML 张量库/OS 多线程/OS IO/Cgroups+Taskset/并发编程/Transformer

(6) Github 代码仓库理解

仓库本身就是 fork 自 llama.cpp，然后改出来的。

①ggml/：底层数学和张量计算库，包含底层数据结构和算子，flex 在内存分配器和文件读取里面基于 direct io 做了文件读取接口。

②src/：llm 的核心逻辑，包含 llama.cpp 文件，定义 transformer 架构、模型权重、kv cache 之类的。Flex 的异步预取（前向传播循环里面，从一层一层改成了 io 线程）、内存锁定（模型初始化加载的时候，计算出 lock_ratio）、张量保留（模型加载和张量内存映射分配阶段，直接 ifelse 做的）都在这里做的。

③common/：通用工具和组件，处理用户输入、构建 sys prompt、采样算法（temperature、top-p），flex 新增了一些参数控制。

④examples/：上层应用，调用 dicengapi 的示例程序，

三、进一步可以做的拔高

1. KV Cache 在 OS 的优化

Flex 其实没有专门面向 KV 的特性，所以可以考虑引入 Paged Attention 换入换出，像赵老师说的那样引入一个进化算法或者训练一个好的策略算法出来，专门测评 Cache 温度。

结合 llm 的注意力分数特性（如 Heavy Hitters 现象），引入注意力权重的系统级启发淘汰。需要深入学习 KV Cache 定义和注意力分数。

3. MoE 预测性加载

我去查了一下 HF 上的模型库，目前 qwen 系列情况如下：

①Qwen3：0.6/1.7/4/8/14/32 B，这几个是 dense 模型。30B-A3B 和 235B-A22B 这俩是 MoE。分别可以开 thinking 和 fp8。

②Qwen3-Next：专门做超稀疏的，80B-A3B 这一种规格，非常适合端侧满血，有 Instruct 和 Thinking，分别可以开 fp8。

③Qwen3.5：0.8/2/4/9/27/35/122/397 的 dense 规格，35-A3/122-A10/397-A17 三个型号的 MoE。有 Base 和 INT4。

④Qwen3.6：型号少，只有 27B/35B-A3B 这两种开源了，但是应该比较强。

端侧的内外存设计其实跟 MoE 比较适配，由于 router 层在前置计算阶段或者上一层里面，所以可以实现一个轻量级路由预测器，提前一两层预测出专家的激活，提前把专家权重取到内存。

4. 横向、纵向的硬件设备拓展（选择性完成，不求全做）

纵向三级：

①极限受限端侧（核心展示区）：树莓派、RK3588 或 昇腾 310P。物理内存极小（8G/16G），只有纯 CPU 或微型 NPU。在这里跑 Qwen3/Qwen2.5 的 0.5B/1.5B/3B Dense 模型。这能完美展现对底层 Page Fault 和多线程 I/O 的极限压榨。

②常规受限端侧（技术拔高区）：13900+64G+4060-8G 笔记本电脑，此时显存（8G）远小于模型体积，系统处于“显存受限，内存充裕，外存极大”的状态。这是展示 Qwen3.6-35B-A3B /Qwen-Next 等 MoE 架构的最佳舞台，用于展示内存分级卸载机制。

③进阶边缘节点：在 4 卡 A100 上做限制，讨论泛化性。

横向：引入对于 RK3588/海光 DCU/昇腾（刚好打编译/智创比赛有环境）这样的环境适配，对国内比赛的 story 很有帮助。

5. 引入 GPU+VRAM（考虑，但不一定真的做）

核心问题：多级存储的带宽漏斗效应与 PCIe 数据颠簸。GPU 的算力远超 CPU，但如果 GPU 要频繁等待数据从 SSD 拷到 RAM，再从 RAM 走 PCIe 拷进 VRAM，GPU 的利用率会暴跌至 10% 以下。

FlexGen (<https://arxiv.org/abs/2303.06865>)

PowerInfer (<https://arxiv.org/abs/2312.12456>)

考虑以下三种方法：

①静态锚定 (Static Anchoring)：将最核心的、激活频率极高的张量（如 MoE 中的共享专家、Dense 模型中的低层权重）永远锁死在 8G VRAM 中。

②RAM 作为高速缓存 (Buffer)：将内存设置为“中转站”。当 GPU 在计算第 i 层时，CPU I/O 线程不仅要从前存预取第 $i+1$ 层的参数到 RAM，还要负责把 RAM 中的第 $i+1$ 层参数通过 PCIe DMA 异步推送到 VRAM 的另一块 Ping-Pong Buffer 中。

③计算与传输的重叠 (Overlap)：这是 OS 调度的精髓，需要精确计算 GPU 的计算耗时与 PCIe 的传输耗时，力求 $T_{\text{compute}} \approx T_{\text{transfer}}$ 。

6. 升级为 Agent 工作流联调，GPU+CPU 的另一个方向探索（在决赛做，不麻烦）

Agent 场景下执行 tool call 时，本地计算资源其实是空闲的，或者 CPU 满载而 GPU 空闲，考虑做 masking 优化。

在发起 tool call 之后，挂起推理计算线程，利用等待 call 返回的免费时间中，全速运行 IO 线程，把即将用到的系统 prompt 的 KVCache 或者预测的专家权重先取到内存。

7. Benchmark、Baseline 与 SOTA

①BaseLine and Sota

Baseline1：原生 llama.cpp+mmap

大 Baseline 或者小 sota：FlexInfer

Sota：可以考虑搜点别的，或者用 PowerInfer-2: <https://arxiv.org/abs/2406.06282>

②Metrics

吞吐量 token/s

首字延迟 (TTFT) 与 单字生成延迟 (TPOT) 的 P90/P99

PFps, 缺页中断率

IO 吞吐和带宽利用率

KV Cache 命中率

8. 可视化

这个优先级比较后了，主要是决赛可以考虑做一个，AI 写一个都行，评审好看很多。

四、相关论文调研（查了最近几个月的，精读）

1. KV Cache 在 OS 的卸载和 IO Masking

①[2604.26557] [DUAL-BLADE: Dual-Path NVMe-Direct KV-Cache Offloading for Edge LLM Inference](#)

Icdcs26 的论文，提出了一种双路径 KV Cache 驻留框架。当内存吃紧时，它直接绕过操作系统的传统文件系统 (Page-cache)，建立了一条 NVMe-direct 路径，将 KV 张量直接映

射到连续的逻辑块地址 (LBA)。引入了自适应的流水线并行，将存储设备的 NVMe I/O 与 GPU/CPU 的 DMA 传输彻底重叠。不仅把预充填 (Prefill) 和解码 (Decode) 延迟最高降低了 42.4%，还将 SSD 利用率提升了 2.2 倍。

② [\[2603.27138\] ScoutAttention: Efficient KV Cache Offloading via Layer-Ahead CPU Pre-computation for LLM Inference](#)

为了解决 KV 卸载引发的 I/O 瓶颈，提出了一种“跨层提前预计算 (Layer-Ahead Pre-computation)”算法。当 GPU 在处理当前层时，它安排 CPU 提前一层 (One-layer-ahead) 计算下一层的注意力权重，并将关键 Token 提前预取回内存/显存。这种设计用廉价的 CPU 计算力成功抵消了 PCIe/外存的 I/O 延迟，实现了 2.1 倍的推理加速。

③ [\[2602.18750\] HillInfer: Efficient Long-Context LLM Inference on the Edge with Hierarchical KV Eviction using SmartSSD](#)

技术亮点：引入了层级化 KV 淘汰与“智能存储设备 (SmartSSD)”的结合。它将轻量级的“Token 重要性评估 (注意力分数计算)”直接卸载到计算型存储驱动器 (CSD) 内部完成，大幅避免了大量无用 KV 数据在 PCIe 总线上的无效搬运。

2. MoE 端侧和软硬协同部署

① [\[2605.27358v1\] MobileMoE: Scaling On-Device Mixture of Experts](#)

非常热乎，前两天刚发出来。Meta 和三星的论文。首个全面针对消费级移动端 CPU/GPU 进行深度优化的 Sub-billion 级别 MoE 推理框架。模型权重被控制在 3GB 以内 (INT4 量化)，以塞入手机 DRAM。开发了专用的“融合 MoE 算子内核 (Fused MoE Kernel)”，提供了极其详尽的端侧 CPU/GPU 异构 Profiling 数据。相比于同等内存占用的密集型 (Dense) 模型，它的 Decode 速度达到了惊人的 2.2 至 3.4 倍。这篇论文的 Benchmark 可以直接作为 MoE 方向的 SOTA 对标。

② [\[2508.12851v3\] Accelerating Edge Inference for Distributed MoE Models with Latency-Optimized Expert Placement](#)

针对搭载 GPU 的异构边缘服务器，设计了一个协作式的 MoE 它利用了 MoE 工作负载的“输入局部性”，在运行时动态迁移专家权重，在多设备间最小化通信成本，不仅平衡了本地内存占用，还将推理延迟降低了 30.6%。专家放置 (Expert Placement) 策略。

③ [\[2510.19366\] MoE-Prism: 通过模型-系统协同设计，解开弹性 MoE 服务的单体专家](#)

搜到还没细看。

3. 前沿理论综述 (可以从这里出发了解更多的相关论文)

① On-device large language models: a survey of model compression and system optimization

<https://github.com/LumosJiang/Awesome-On-Device-LLMs>

② [\[2603.20397\] KV Cache Optimization Strategies for Scalable and Efficient LLM Inference](#)

将目前的 KV Cache 优化手段严密地分成了五大流派：缓存淘汰、缓存压缩、混合内存方案 (Hybrid Memory Solutions)、新型注意力机制、以及组合策略，操作系统级换入换出方案可以直接在“混合内存方案”这个流派中找到所处的坐标。

五、学习路线规划

1. 大模型推理

①Transformer 深度理解

以下两个挑一个看，或者细看第一个然后加速跳着看第二个。

【【Transformer】最强动画讲解！目前 B 站最全最详细的 Transformer 教程，2025 最新版！从理论到实战，通俗易懂解释原理，草履虫都学的会！】

https://www.bilibili.com/video/BV1fj6vBfEnu/?share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f

【 Transformer 论文逐段精读 【 论文精读 】 】
https://www.bilibili.com/video/BV1pu411o7BE/?share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f

②KVCache

【 什么是缓存命中？8 分钟说清楚 KVCache 与 Prompt Cache 】
https://www.bilibili.com/video/BV1keLp6GERQ/?share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f

【 什么是 KV Cache？为什么它能加快模型推理速度？ 】
https://www.bilibili.com/video/BV1SmrSBfEg6/?share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f

看完之后可以看看 Paged/Flash Attention 和计算

【 什么是 PagedAttention？为什么它能提高 GPU 显存的使用效率？ 】
https://www.bilibili.com/video/BV1vKzuBmEWk/?share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f

【 GPU 是如何一步步计算 Transformer 中的自注意力机制的？ 】
https://www.bilibili.com/video/BV1zFFTziEdL/?share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f

【 什么是 FlashAttention？为什么它能大大减少显存的需要并加快速度？ 】
https://www.bilibili.com/video/BV1irPTzGEhG/?share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f

③了解 MoE

三个挑一个看。

【MoE 架构：20 分钟带你解析 MoE 混合专家模型！MoE 架构深度拆解,全程干货！大模型 | LLM】
https://www.bilibili.com/video/BV1wPUDBYErR/?share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f

【 认识混合专家模型（MoE） 】
https://www.bilibili.com/video/BV1Ep421m7cx/?share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f

【 MOE 终于迎来可视化解读！傻瓜都能看懂 MoE 核心原理！ 】

https://www.bilibili.com/video/BV1Gj9ZYdE4N/?share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f

④模型压缩

【【人工智能】模型压缩四大方法概述 | 量化、剪枝、蒸馏和二值化 | 模型瘦身 | 降低精度 | 速度提升 | 知识蒸馏 | 温度参数 | XNOR | 优缺点】
https://www.bilibili.com/video/BV1MFPRreTECW/?share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f

【深度学习模型压缩概述】
https://www.bilibili.com/video/BV1ht4y1P73i/?share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f

剪枝 torchpruning:

<https://zhuanlan.zhihu.com/p/619482727>

https://blog.csdn.net/gitblog_09509/article/details/142231614

张量学习 tensorly:

<https://zhuanlan.zhihu.com/p/681899951>

<https://www.dongaigc.com/a/tensorly-powerful-python-tensor-librar>

量化:

【【大语言模型】量化技术详解 | 常见量化方法 | PTQ | QAT】
https://www.bilibili.com/video/BV1iyNmzxECP/?share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f

2. C++与高性能

①并行程设课内的相关内容学完。

②llama.cpp, 边看边学一下高性能 C++

<https://github.com/ggml-org/llama.cpp>

<https://www.aidoczh.com/llama-cpp/docs/intro/>

[理解 llama.cpp 怎么完成大模型推理的 - 知乎](#)

[深入解析：解密 llama.cpp：从 Prompt 到 Response 的完整技术流程剖析 - yxysuanfa - 博客园](#)

[llama.cpp 源码解析 llama.cpp-CSDN 博客](#)

这个系列挑着看:

【【大模型部署】llama.cpp 源码逐行调试带读！（已完结~）】
https://www.bilibili.com/video/BV1N4wreWE8z/?share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f

3. Linux OS 内存和 IO

①复习虚存和缺页

【【硬核】操作系统最难啃的骨头？动画演示 MMU 与 TLB 工作原理，看完通透！| 页表架构 / 页号 / 页内偏移 / 物理内存 / TLB 缓存 / 缺页中断】
https://www.bilibili.com/video/BV1pjqnBXEbQ/?share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f

【用动画讲讲 mmap & pagefault】
https://www.bilibili.com/video/BV17V411N7jy/?share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f

②高级 IO 与异步 IO

<https://zhuanlan.zhihu.com/p/374627314>

<https://blog.csdn.net/hilaryfrank/article/details/112200386>

<https://zhuanlan.zhihu.com/p/112162403>

<https://zhuanlan.zhihu.com/p/1934342933789806664>

【到底什么是 io_uring】
https://www.bilibili.com/video/BV1JitoeXErR/?share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f

③资源隔离与限制

【【小神仙讲 docker】| 底层原理 | 通俗易懂 | namespace | cgroups | 文件系统 | 佩雨小神仙】
https://www.bilibili.com/video/BV1oi4y1F7sb/?p=2&share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f

浅谈 Linux Cgroups 机制 - lecury 的文章 - 知乎 <https://zhuanlan.zhihu.com/p/81668069>

4. MLSys 和 AI Infra

①infra 入门

【AI Infra 10 分钟入门：Triton, TVM, XLA, Alpa...?】
https://www.bilibili.com/video/BV1Hz33z1EK6/?share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f

②计算图

【为什么 AI 框架都用计算图？什么是计算图？到底计算图有什么用？【计算图】系列第二篇】
https://www.bilibili.com/video/BV1rR4y197HM/?share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f

③张量

【为什么 AI 框架都用计算图？什么是计算图？到底计算图有什么用？【计算图】系列第二篇】
https://www.bilibili.com/video/BV1rR4y197HM/?share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f

④算子融合

【从零到一：像刷力扣一样手撕 100 个 AI Infra 的 Kernel 算子（CUDA+Triton+Pytorch）】

https://www.bilibili.com/video/BV1qhDXBzEbA/?share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f

关于算子优化可以看：【斯坦福 CS336 第六课：手写高性能算子】

https://www.bilibili.com/video/BV1PPEkzxEmW/?share_source=copy_web&vd_source=7b93c3a0d0bdbab6f0e449df79db946f